

G21 / Linux commit 97cba232: off-by-one look-ahead access

A model-produced capacity check repairs a fixed-side false alarm

→ control flow
---> checker-to-code link

#A. Patch and declared array extent

M1 enum { N = MAX_PIPES * 2 }; // figure alias

M2 struct dc_link *links[N];

M3 // get_host_router_total_dp_tunnel_bw(...)

M4 ...

M5 - for (uint8_t i=0; i<N; ++i) {

M6 + for (uint8_t i=0; i<N-1; ++i) {

M7 if (!dc->links[i] || ...) continue;

M8 ...

M9 if (hr_index_temp == hr_index) {

M10 primary = dc->links[i];

M11 secondary=dc->links[i+1];

M12 ...;

M13 break;

M14 }

M15 }

1 capacity N

2 bound B = N

3 look-ahead i+1

4 bound B = N-1

Vulnerable: $\max(i+1) = N$ (out of bounds)

Fixed: $\max(i+1) = N - 1$ (in bounds)

#B. Before refinement

B1 checkASTCodeBody(F):

B2 for each recognized loop $i < B$:

B3 find look-ahead access $A[i + 1]$

B4 if no recognized body guard:

B5 report($A[i + 1]$)

B6 // Missing: compare loop reach

B7 // with the declared capacity of A

Linux V/F: 1/1

5 missing bound proof

#C. After automatic refinement

C1 checkASTCodeBody(F): // retained

C2 retain loop and access discovery

C3 $N = \text{declared capacity of } A$

C4 $\max_index = (B - 1) + 1$

C5 if $\max_index < N$ is established:

C6 suppress the false alarm

C7 otherwise retain the warning

C8 // normalize supported guards/tests

C9 // no patch-name or line allowlist

Linux V/F: 1/0

6 capacity-aware predicate

Solid arrows: patch control flow. Dashed arrows: semantic links. Checker panels are abridged logic.

Frozen candidate + modeltrace + paired execution + perturbation results are supplied with the figure.